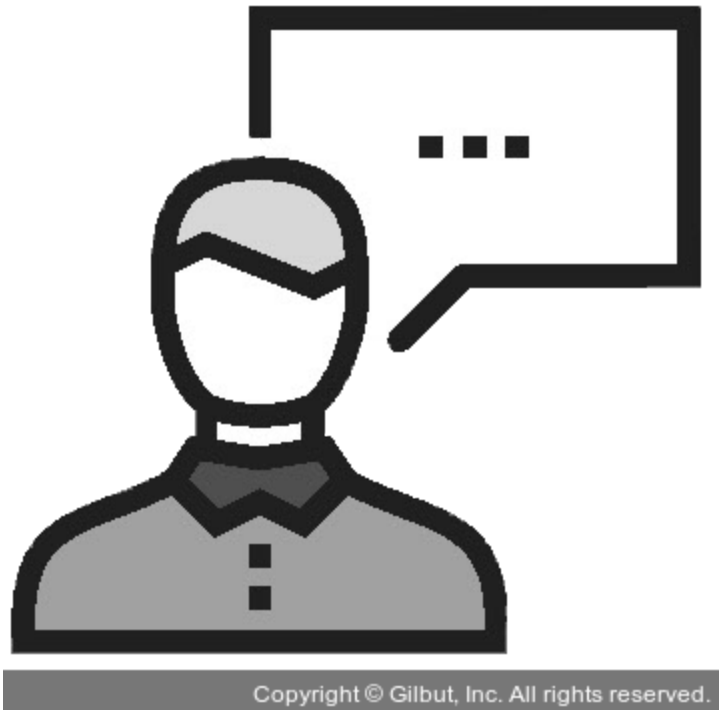
UNIT 05
내 생일의 기온 변화를 그래프로 그리기



⦿ 실습 데이터 Unit05/unit05-01.ipynb, seoul.csv

1 데이터에 질문하기

여러분의 생일은 언제인가요? 그날 날씨는 어땠을까요? 시원한 바람이 부는 맑은 날씨였나요, 아니면 기록적인 한파가 기승을 부리던 날이었나요? 여러분이 태어난 날부터 지금까지 매년 생일의 최고 기온을 그래프로 그린다면 어떤 모양일까요? 만약 지금까지 생일의 최고 기온을 꺾은선 그래프로 표현하면 기온이 계속 상승하는 모양일까요?

이 장에서는 데이터에서 발견한 이런 질문들을 서울의 기온 공공데이터를 활용해서 해결해 보겠습니다.

■ 데이터 읽어오기

먼저 서울의 기온 데이터를 읽어와서 한 줄씩 출력하겠습니다.

```
import csv
f = open('seoul.csv')
data = csv.reader(f)
for row in data :
    print(row)
```

TIP

Unit 5부터는 실행 결과에 영향을 주지 않을 경우 f.close( ) 함수를 생략하겠습니다.

실행 결과

```
['날짜', '지점', '평균기온(℃)', '최저기온(℃)', '최고기온(℃)']
['1907-10-01', '108', '13.5', '7.9', '20.7']
['1907-10-02', '108', '16.2', '7.9', '22']
['1907-10-03', '108', '16.2', '13.1', '21.3']
['1907-10-04', '108', '16.5', '11.2', '22']
(생략)
```



잠깐만요

맥이나 리눅스 운영체제를 사용하고 있나요?

만약 여러분의 운영체제가 윈도가 아닌 맥이나 리눅스라면 Unit05-01 코드 둘째 줄에 다음과 같이 인코딩 관련 코드를 추가하세요. Unit 2에서 보았듯이 cp949는 Windows 한글 인코딩 방식을 의미합니다. 따라서 윈도가 아닌 맥이나 리눅스라면 한글 인코딩 방식을 지정해야 한글이 깨지지 않고 출력됩니다.

```
f = open('seoul.csv', encoding = 'cp949')
```

■ 데이터 리스트에 저장하기

이제 최고 기온 데이터를 저장하겠습니다. 최고 기온 데이터를 날짜 순으로 저장하고자 `result`라는 리스트를 하나 만듭니다. 그리고 최고 기온 데이터가 존재하는 날의 최고 기온 데이터를 `result` 리스트에 저장합니다.

```
import csv
f = open('seoul.csv')
data = csv.reader(f)
next(data)
result = []                          # 최고 기온 데이터를 저장할 리스트 생성

for row in data :
    if row[-1] != '' :               # 최고 기온 데이터 값이 존재한다면
        result.append(float(row[-1])) # result 리스트에 최고 기온 값 추가
print(result)
```

실행 결과

```
[20.7, 22.0, 21.3, 22.0, 25.4, 21.3, 16.1, 14.9, 21.1, 24.1, 20.4, 17.4, 21.3, 20.6, 20.9, 20.2, 2
1.6, 20.9, 21.3, 22.7, 19.9, ... (생략)]
```

2 데이터 시각화하기

이제 최고 기온 데이터를 꺾은선 그래프로 표현해 보겠습니다. 그래프를 그리려면 먼저 `matplotlib` 라이브러리를 임포트해야 합니다. 그런 다음 `plot()` 함수에 데이터를 넣고 `show()` 함수로 보여주면 완성입니다. 최고 기온이니 '뜨거움'을 나타내는 빨간색으로 그래프를 표현해 볼까요?

```
import csv
import matplotlib.pyplot as plt
(생략)
plt.plot(result, 'r')    # result 리스트에 저장된 값을 빨간색 그래프로 그리기
plt.show()               # 그래프 나타내기
```

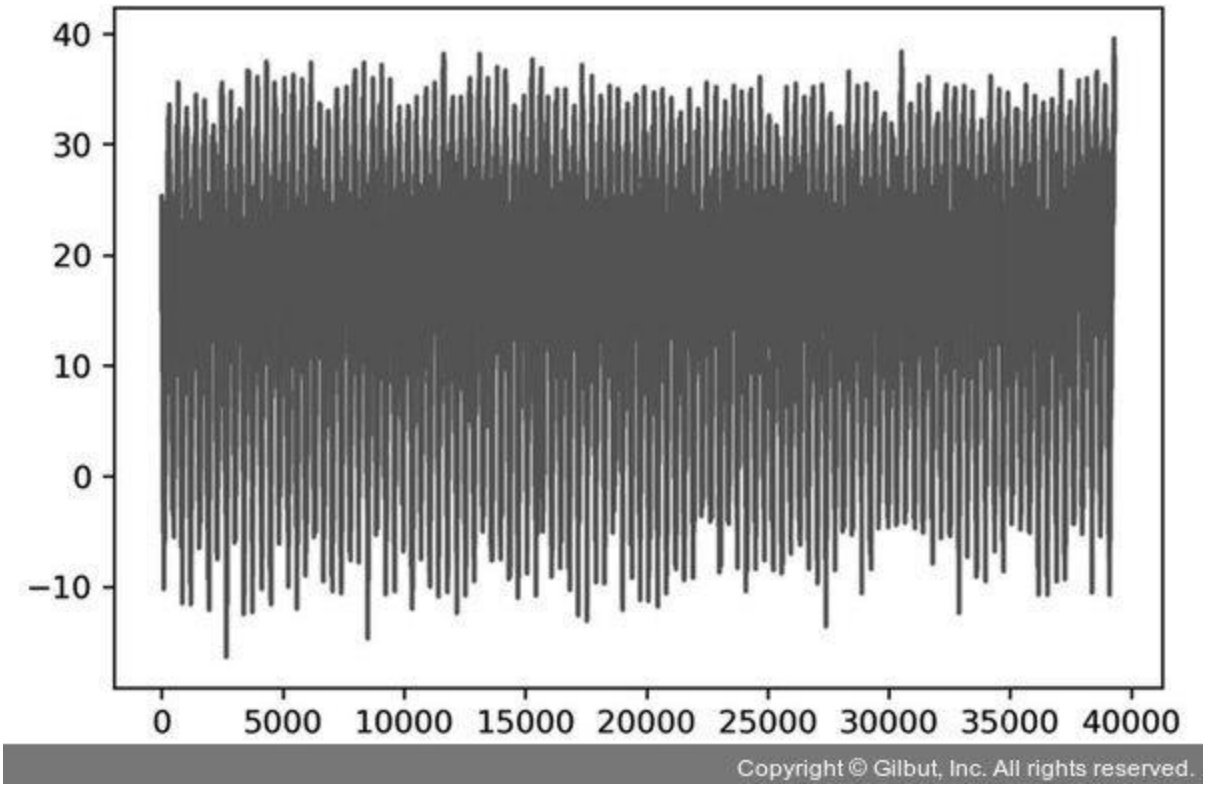


그림 5-1 최고 기온 데이터를 꺾은선 그래프로 나타낸 모습

3 날짜 데이터 추출하기

파이썬이 제공하는 문자열 함수 중 `split()`는 사용자가 설정하는 특정 문자를 기준으로 문자열을 분리합니다. 다음 예시 코드를 볼까요?

```
s = 'hello python'
print(s.split())
```

실행 결과

```
['hello', 'python']
```

코드를 실행하면 `['hello', 'python']`이라는 리스트가 출력됩니다. 여기서 알 수 있듯이 `split()` 함수는 사용자가 설정하는 특정 문자가 없다면 기본적으로 공백 문자를 기준으로 문자열을 분리합니다.